

Exp No.: 10 Troubleshooting Fabric Applications

Aim

To identify, analyze, and resolve common issues encountered in Hyperledger Fabric applications by troubleshooting network connectivity, chaincode deployment, channel configuration, certificate authentication, and Docker container errors.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Theory

Troubleshooting is an essential part of administering Hyperledger Fabric networks. Errors may occur due to incorrect configuration, network failures, certificate issues, chaincode deployment failures, or Docker container problems. Common troubleshooting techniques include:

- Checking Docker container status
- Viewing peer and orderer logs
- Verifying channel membership
- Checking chaincode installation
- Validating MSP and certificates
- Restarting Fabric services
- Cleaning Docker resources

Proper troubleshooting minimizes downtime and ensures reliable blockchain application deployment.

Figure 1: Anchor peer configuration completed and Docker containers running

Step 4: Check Orderer Logs

Inspect orderer logs for Raft consensus and block processing.

```
docker logs --tail 30 orderer.example.com
```

```
(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ docker logs --tail 30 orderer.example.com
2026-09-08 18:06:06.398 UTC 001f INFO [orderer.consensus.etcdraft] switchToConfig → 1 switched to configuration voters=(1)
channel=mychannel node=1
2026-09-08 18:06:06.398 UTC 0020 INFO [orderer.consensus.etcdraft] apply → Applied config change to add node 1, current no
des in channel: [1] channel=mychannel node=1
2026-09-08 18:06:07.394 UTC 0021 INFO [orderer.consensus.etcdraft] hup → 1 is starting a new election at term 1 channel=my
channel node=1
2026-09-08 18:06:07.394 UTC 0022 INFO [orderer.consensus.etcdraft] becomePreCandidate → 1 became pre-candidate at term 1 c
hannel=mychannel node=1
2026-09-08 18:06:07.394 UTC 0023 INFO [orderer.consensus.etcdraft] poll → 1 received MsgPreVoteResp from 1 at term 1 chann
el=mychannel node=1
2026-09-08 18:06:07.394 UTC 0024 INFO [orderer.consensus.etcdraft] stepCandidate → 1 has received 1 MsgPreVoteResp votes a
nd 0 vote rejections channel=mychannel node=1
2026-09-08 18:06:07.394 UTC 0025 INFO [orderer.consensus.etcdraft] becomeCandidate → 1 became candidate at term 2 channel=
mychannel node=1
2026-09-08 18:06:07.398 UTC 0026 INFO [orderer.consensus.etcdraft] poll → 1 received MsgVoteResp from 1 at term 2 channel=
mychannel node=1
2026-09-08 18:06:07.398 UTC 0027 INFO [orderer.consensus.etcdraft] stepCandidate → 1 has received 1 MsgVoteResp votes and
0 vote rejections channel=mychannel node=1
2026-09-08 18:06:07.398 UTC 0028 INFO [orderer.consensus.etcdraft] becomeLeader → 1 became leader at term 2 channel=mychan
nel node=1
2026-09-08 18:06:07.398 UTC 0029 INFO [orderer.consensus.etcdraft] run → raft.node: 1 elected leader 1 at term 2 channel=m
ychannel node=1
2026-09-08 18:06:07.399 UTC 002a INFO [orderer.consensus.etcdraft] run → Leader 1 is present, quit campaign channel=mychan
nel node=1
2026-09-08 18:06:07.400 UTC 002b INFO [orderer.consensus.etcdraft] run → Raft leader changed: 0 → 1 channel=mychannel nod
e=1
2026-09-08 18:06:07.400 UTC 002c INFO [orderer.consensus.etcdraft] run → Start accepting requests as Raft leader at block
[0] channel=mychannel node=1
2026-09-08 18:06:12.749 UTC 002d WARN [common.deliver] Handle → Error reading from 172.18.0.1:55270: rpc error: code = Can
celed desc = context canceled
2026-09-08 18:06:12.749 UTC 002e INFO [comm.grpc.server] 1 → streaming call completed grpc.service=orderer.AtomicBroadcast
grpc.method=Deliver grpc.peer_address=172.18.0.1:55270 error="rpc error: code = Canceled desc = context canceled" grpc.cod
e=Canceled grpc.call_duration=8.716513ms
2026-09-08 18:06:13.017 UTC 002f WARN [orderer.consensus.etcdraft] checkForEvictionNCertRotation → could not read config m
etadata: %!(s<nil>) channel=mychannel node=1
2026-09-08 18:06:13.017 UTC 0030 INFO [orderer.consensus.etcdraft] propose → Created block [1], there are 0 blocks in flig
ht channel=mychannel node=1
```

Figure 2: Orderer logs — Raft leader election, block creation, and transaction processing

Step 5: Verify Channel Membership

```
peer channel list
```

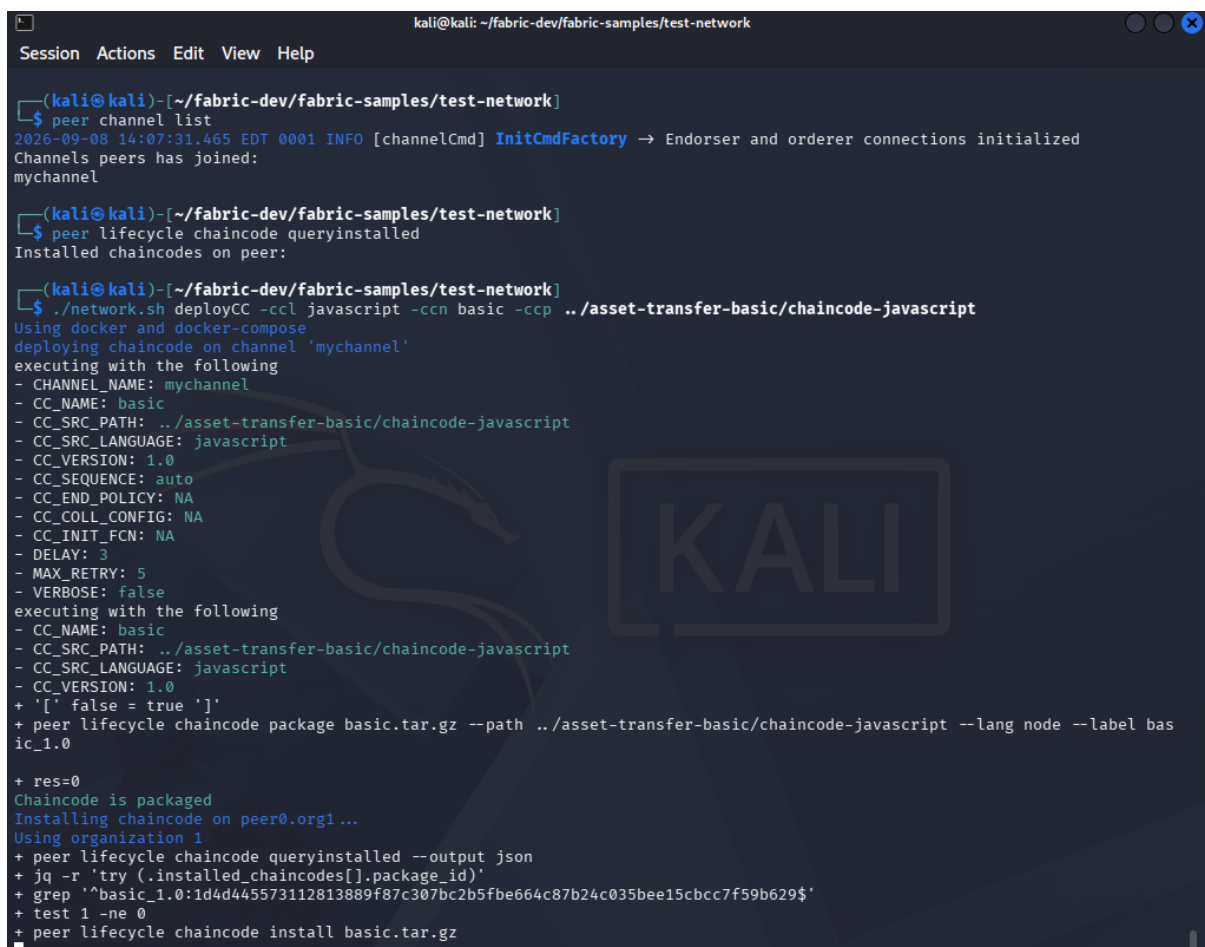
Step 6: Verify Installed Chaincode

```
peer lifecycle chaincode queryinstalled
```

Step 7: Troubleshoot Chaincode Deployment

Redeploy the chaincode if deployment has failed.

```
./network.sh deployCC -ccl javascript -ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript
```



```
kali@kali: ~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer channel list
2026-09-08 14:07:31.465 EDT 0001 INFO [channelCmd] InitCmdFactory → Endorser and orderer connections initialized
Channels peers has joined:
mychannel

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer lifecycle chaincode queryinstalled
Installed chaincodes on peer:

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ ./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[0].package_id)'
+ grep '^basic_1.0:1d4d445573112813889f87c307bc2b5f5be664c87b24c035bee15cbcc7f59b629$'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
```

Figure 3: Channel verification, chaincode query, and chaincode redeployment

Step 8: Verify Chaincode Approval and Commitment

Check approval status across both organizations.

```
peer lifecycle chaincode checkcommitreadiness --channelID mychannel \
--name basic --version 1.0 --sequence 1 --output json
```

Step 9: Check Docker Resource Usage

Monitor CPU and memory utilization of containers.

```
docker stats --no-stream
```

```

kali@kali: ~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help
    "Org1MSP": true,
    "Org2MSP": false
  }
}
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'mychannel'
Using organization 2
+ peer lifecycle chaincode approveformyorg -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile
e /home/kali/fabric-dev/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-
cert.pem --channelID mychannel --name basic --version 1.0 --package-id basic_1.0:1d4d445573112813889f87c307bc2b5f6e664c87b2
4c035bee15cbcc7f59b629 --sequence 1
+ res=0
2026-09-08 14:08:45.230 EDT 0001 INFO [chaincodeCmd] ClientWait -> txid [954ca17c025456975825800bb9121b48d35ca87a791ce6e03c
636b0183f33f68] committed with status (VALID) at localhost:9051
Chaincode definition approved on peer0.org2 on channel 'mychannel'
Using organization 1
Checking the commit readiness of the chaincode definition on peer0.org1 on channel 'mychannel' ...
Attempting to check the commit readiness of the chaincode definition on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json
+ res=0
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": true
  }
}
Checking the commit readiness of the chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Checking the commit readiness of the chaincode definition on peer0.org2 on channel 'mychannel' ...
Attempting to check the commit readiness of the chaincode definition on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json
+ res=0
{
  "approvals": {
CONTAINER ID   NAME
% MEM USAGE / LIMIT     MEM %    NET I/O    BLOCK I/O  PIDS
9ce4e7a1f4b5   dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f6e664c87b24c035bee15cbcc7f59b629 0.00
% 49.71MiB / 2GiB        2.43%    319kB / 10.4kB 217kB / 4.1kB 24
c0051b0e04ec   dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f6e664c87b24c035bee15cbcc7f59b629 0.00
% 50.68MiB / 2GiB        2.47%    320kB / 10.9kB 254kB / 4.1kB 24
bd363c908dce   orderer.example.com
% 20.63MiB / 3.825GiB    0.53%    80.9kB / 254kB 20.5kB / 266kB 9
93411d60e273   peer0.org2.example.com
% 108.1MiB / 3.825GiB    2.76%    394kB / 268kB  0B / 537kB  9
4b72e20e9207   peer0.org1.example.com
% 118.7MiB / 3.825GiB    3.03%    407kB / 291kB  0B / 537kB  10
f34ffb637399   ca_orderer
% 19.34MiB / 3.825GiB    0.49%    52.3kB / 84.6kB 4.63MB / 1.05MB 9

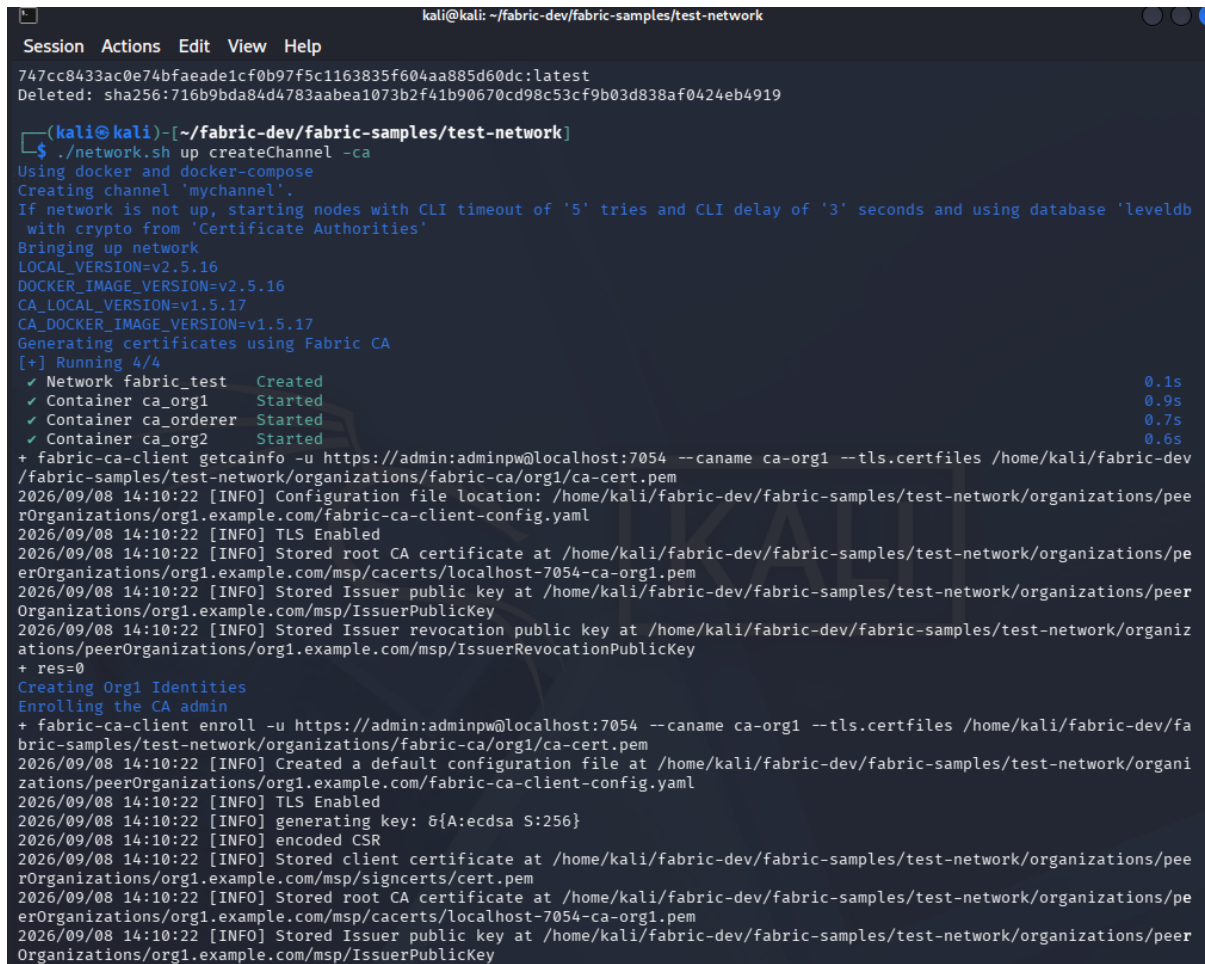
```

Figure 4: Chaincode approval, commit readiness (both orgs true), and container resource stats

Step 10: Restart the Fabric Network

Stop and restart the network to verify clean recovery.

```
./network.sh down  
./network.sh up createChannel -ca
```



```
kali@kali: ~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help
747cc8433ac0e74bfaeade1cf0b97f5c1163835f604aa885d60dc:latest
Deleted: sha256:716b9bda84d4783aabea1073b2f41b90670cd98c53cf9b03d838af0424eb4919

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ ./network.sh up createChannel -ca
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb'
with crypto from 'Certificate Authorities'
Bringing up network
LOCAL_VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
CA_LOCAL_VERSION=v1.5.17
CA_DOCKER_IMAGE_VERSION=v1.5.17
Generating certificates using Fabric CA
[+] Running 4/4
  ✓ Network fabric_test      Created                                0.1s
  ✓ Container ca_org1        Started                               0.9s
  ✓ Container ca_orderer     Started                               0.7s
  ✓ Container ca_org2        Started                               0.6s
+ fabric-ca-client getcainfo -u https://admin:adminpw@localhost:7054 --caname ca-org1 --tls.certfiles /home/kali/fabric-dev/
fabric-samples/test-network/organizations/fabric-ca/org1/ca-cert.pem
2026/09/08 14:10:22 [INFO] Configuration file location: /home/kali/fabric-dev/fabric-samples/test-network/organizations/pee
rOrganizations/org1.example.com/fabric-ca-client-config.yaml
2026/09/08 14:10:22 [INFO] TLS Enabled
2026/09/08 14:10:22 [INFO] Stored root CA certificate at /home/kali/fabric-dev/fabric-samples/test-network/organizations/pee
rOrganizations/org1.example.com/msp/cacerts/localhost-7054-ca-org1.pem
2026/09/08 14:10:22 [INFO] Stored Issuer public key at /home/kali/fabric-dev/fabric-samples/test-network/organizations/peer
Organizations/org1.example.com/msp/IssuerPublicKey
2026/09/08 14:10:22 [INFO] Stored Issuer revocation public key at /home/kali/fabric-dev/fabric-samples/test-network/organiz
ations/peerOrganizations/org1.example.com/msp/IssuerRevocationPublicKey
+ res=0
Creating Org1 Identities
Enrolling the CA admin
+ fabric-ca-client enroll -u https://admin:adminpw@localhost:7054 --caname ca-org1 --tls.certfiles /home/kali/fabric-dev/fa
bric-samples/test-network/organizations/fabric-ca/org1/ca-cert.pem
2026/09/08 14:10:22 [INFO] Created a default configuration file at /home/kali/fabric-dev/fabric-samples/test-network/organi
zations/peerOrganizations/org1.example.com/fabric-ca-client-config.yaml
2026/09/08 14:10:22 [INFO] TLS Enabled
2026/09/08 14:10:22 [INFO] generating key: 8{A:ecdsa S:256}
2026/09/08 14:10:22 [INFO] encoded CSR
2026/09/08 14:10:22 [INFO] Stored client certificate at /home/kali/fabric-dev/fabric-samples/test-network/organizations/pee
rOrganizations/org1.example.com/msp/signcerts/cert.pem
2026/09/08 14:10:22 [INFO] Stored root CA certificate at /home/kali/fabric-dev/fabric-samples/test-network/organizations/pee
rOrganizations/org1.example.com/msp/cacerts/localhost-7054-ca-org1.pem
2026/09/08 14:10:22 [INFO] Stored Issuer public key at /home/kali/fabric-dev/fabric-samples/test-network/organizations/peer
Organizations/org1.example.com/msp/IssuerPublicKey
```

Figure 5: Network restart — CA containers started, certificates generated successfully

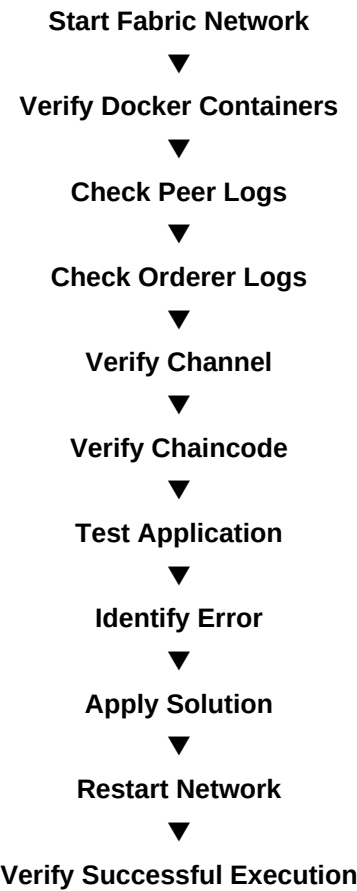
Step 11: Clean Docker Resources (Optional)

```
docker system prune -f
```

Common Errors and Solutions

Error Message	Possible Cause	Solution
Docker daemon not running	Docker service stopped	sudo systemctl start docker
Peer connection failed	Peer container stopped	Restart Fabric network
Chaincode deployment failed	Incorrect chaincode path	Verify -ccp directory
Channel not found	Channel not created	Run ./network.sh createChannel
Access denied	Invalid MSP or certificates	Regenerate certificates
TLS handshake failed	Incorrect TLS certificate	Verify TLS CA files
Port already in use	Another service using port	Stop conflicting process

Troubleshooting Workflow



Result

The Hyperledger Fabric application was successfully analyzed and common operational issues were identified and resolved using Docker commands, Fabric CLI tools, and log analysis. The experiment demonstrated effective troubleshooting techniques for network connectivity, chaincode deployment, channel configuration, certificate management, and resource monitoring.